

UNIT 3

Syllabus: Stacks: Definition, ADT, standard stack operations- array and linked list implementations, applications - infix to postfix conversion, postfix expression evaluation, parsing parenthesis, reverse of a string using stack, history of a browser, etc

Queues: Definition, ADT, standard queue operations - array and linked implementations, Circular queues - Insertion and deletion operations. Real Time Applications of Queue: Operating Systems and Task Scheduling, Networking and Message Queues, etc

=====

Stack Data Structure

Stack is a linear data structure that operates in a LIFO (Last In First Out) FILO (First In Last Out) pattern.

- It is named stack as it behaves like a real-world stack.
Example: A pile of chairs
- Stack is an abstract data type with a bounded (predefined) capacity.
- It is a simple data structure that allows adding and removing elements in a particular order.
- The order may be LIFO or FILO.

Operations on stack

Push(): A data element is inserted into the stack.

Pop(): A data element is removed from the stack.

isEmpty(): The function says that elements are in the stack present or no elements.

isFull(): It says that the stack is full of elements or not.

Peek(): Access the element from a required position.

Count(): Counts the number of elements from the stack.

Change(): Changes the particular element of the stack.

Display(): It prints all the elements of the stack.

Applications of Stack data structure

- Balancing of symbols
- Conversion of Postfix into Prefix vice versa.
- Redo-undo features at many places like editors, photoshop.
- Forward and backward feature in web browsers.
- Used in many algorithms like Towers of Hanoi, Knight tour problem, rat in a maze N – queen and sudoku solver.
- In graph algorithm like Topological Sorting and Strongly Connected Components.

Variable: Top holds the location of the last element that enters into the stack.

Adding an element onto the stack (push operation)

Adding an element into the top of the stack is referred to as push operation. Push operation involves following two steps.

Increment the variable Top so that it can now refer to the next memory location.

Add element at the position of incremented top. This is referred to as adding new element at the top of the stack.

Stack is overflown when we try to insert an element into a completely filled stack therefore, our main function must always avoid stack overflow condition.

Deletion of an element from a stack (Pop operation)

Deletion of an element from the top of the stack is called pop operation. The value of the variable top will be incremented by 1 whenever an item is deleted from the stack. The top most element of the stack is stored in an another variable and then the top is decremented by 1. the operation returns the deleted value that was stored in another variable as the result.

The underflow condition occurs when we try to delete an element from an already empty stack.

A C++ program that implements the stack operations using an array

```
#include<iostream>
#define SIZE 5

using namespace std;

class Stack
{
    public:
    int top = -1;
    int arr[SIZE];

    void push (int val)
    {
        if (top == SIZE)
        {
            cout << "Stack overflow" << endl;
        }
        else
        {
            top++;
            arr[top] = val;
        }
    }

    int pop()
    {
        if (top == -1)
        {
            cout << "Stack underflow" << endl;
            exit(1);
        }
        else
```

```

        {
            int popValue = arr[top];
            arr[top] = 0;
            top--;
            return popValue;
        }
    }

int count()
{
    return (top + 1);
}

int peek(int pos)
{
    if (top == -1)
    {
        cout << "Stack underflow" << endl;
        exit(1);
    }
    else
    {
        return arr[pos];
    }
}

void change(int pos, int val)
{
    arr[pos] = val;
    cout << "Value changed at location " << pos << endl;
}

void display()
{
    cout << "Elements in the Stack are ...";
    for (int i = SIZE; i >= 0; i--)
    {
        cout << arr[i] << " ";
    }
    cout<<endl;
}
};

int main()
{
    Stack s1;
    int option, postion, value;

    cout << "1. Push()" << endl;
    cout << "2. Pop()" << endl;

```

```
cout << "3. display()" << endl;
cout << "4. count()" << endl;
cout << "5. change()" << endl;
cout << "6. Peek()" << endl;
```

```
do
{
    cout << endl << "Choose the below to continue or 0 to exit ...";
    cin >> option;
    switch (option)
    {
        case 1:
            cout << "Enter an item to push in the stack ...";
            cin >> value;
            s1.push(value);
            break;

        case 2:
            cout << "Popped Element ...";
            cout << s1.pop() << endl;
            break;

        case 3:
            s1.display();
            break;

        case 4:
            cout << "Number of elements in the Stack are: ";
            cout << s1.count() << endl;
            break;

        case 5:
            cout << "Enter position of the element to be changed ... ";
            cin >> postion;
            cout << "Enter a new element ... ";
            cin >> value;
            s1.change(postion, value);
            break;

        case 6:
            cout << "Enter position of element you want to see ... ";
            cin >> postion;
            cout << "Value at position is ... ";
            cout << s1.peek(postion) << endl;
            break;

        default:
            cout << "Enter Correct Option number " << endl;
    }
}
```

```
} while (option != 0);
```

```
    return 0;  
}
```

Implementation of Stack using Linked List:

A linked list is a linear data structure consisting of nodes, where each node contains data and a reference (link) to the next node in the sequence. The last node typically points to null or None, indicating the end of the list.

In the context of a stack implemented with a linked list, each node represents an element in the stack.

Each node contains two components: data to store the actual element, and next to reference the next node in the sequence.

Stack Class

The stack class has a reference to the top node.

The push operation adds a new node to the top of the stack.

The pop operation removes the top node from the stack.

The peek operation returns the data of the top node without removing it.

The isEmpty operation checks if the stack is empty.

Stack Implementation using linked list

```
#include <iostream>  
using namespace std;
```

```
class Node  
{  
    public:  
        int data;  
        Node *link;  
};  
  
class Stack  
{  
    public:  
        Node *top = NULL;  
  
        void push (int value)  
        {  
            Node *ptr = new Node();  
            ptr->data = value;  
            ptr->link = top;  
            top = ptr;  
        }  
  
        void pop ( )  
        {  
            if ( top == NULL )
```

```

        cout<<"Stack is Empty";
    else
    {
        cout<<"The deleted element ..."<<top->data<<endl;
        top = top -> link;
    }
}

```

```

void peek()
{
    if ( top == NULL )
        cout<<"Stack is Empty"<<endl;
    else
        cout<<"Element at top ... "<< top->data<<endl;
}

```

```

void display()
{
    if (top == NULL)
        cout<<"Stack is Empty"<<endl;
    else
    {
        Node *temp=top;

        while(temp!=NULL)
        {
            cout<<temp->data<<"-->";
            temp=temp->link;
        }
        cout<<endl;
    }
}

```

```

};

```

```

int main()

```

```

{
    Stack s;
    int option, value;

```

```

        cout<<"1. Push"<<endl;
        cout<<"2. Pop"<<endl;
        cout<<"3. Peek"<<endl;
        cout<<"4. Display"<<endl;
        cout<<"5. Exit"<<endl;

```

```

do
{

```

```

    cout<<"Enter an option ...";
    cin>>option;

    switch (option)
    {
        case 1: cout<<"Enter Value ...";
                cin>>value;
                s.push(value);
                break;

        case 2: cout<<endl;
                s.pop();
                break;

        case 3: cout<<endl;
                s.peek();
                break;

        case 4: cout<<"Elements in the stack ...";
                s.display();
                break;

        case 5: exit(0);
                break;
        default:
            cout << "Enter Correct Option number " << endl;
    }
    }while(option != 0);
    return 0;
}

```

Infix to Postfix Conversion

Infix expressions are mathematical expressions where operators are written between operands, as in conventional mathematical notation. On the other hand, postfix expressions, also known as Reverse Polish Notation (RPN), place the operators after their operands.

Converting an infix expression to postfix involves rearranging the expression to maintain the order of operations without the need for parentheses. The algorithm for this conversion can be implemented using a stack data structure.

Algorithm

- Initialize an empty stack to hold operators.
- Scan the infix expression from left to right.
- For each symbol in the infix expression:
 - If it is an operand, output it directly.
 - If it is an operator:
 - While the stack is not empty and the precedence of the top operator is greater than or equal to the current operator, pop the top operator from the stack and output it.
 - Push the current operator onto the stack.

If it is an open parenthesis '(', push it onto the stack.

If it is a close parenthesis ')':

Pop and output operators from the stack until an open parenthesis '(' is encountered. Pop and discard the '('.

After scanning the entire expression, pop and output any remaining operators from the stack.

Example:

Let's consider the infix expression: $A + B * C - D / E$

Stack: empty, Output: empty

A: Output A

+: Push +

B: Output B

*: Push *

C: Output C

-: Pop * (higher precedence) and output *, then push -

D: Output D

/: Pop - (same precedence) and output -, then push /

E: Output E

End: Pop / and output /, then pop + and output +

Postfix Result: $A B C * + D E / -$

Explanation

The stack helps maintain the order of operations by storing operators.

Operators with higher precedence are popped before pushing a new operator onto the stack.

Parentheses control the order of evaluation; they are pushed onto the stack and popped when a matching close parenthesis is encountered.

This algorithm ensures that the postfix expression, when evaluated, produces the same result as the original infix expression. In postfix notation, there is no need for parentheses, making the expression unambiguous and easier to evaluate.

C++ program to convert infix expression to postfix

Sample Test case:

Enter expression

input= $a+b*c/d-e$

output=

Postfix form: $abc*d/+e-$

```
#include<iostream>
```

```
#include<stack>
```

```
#include<string>
```

```
using namespace std;
```

```
class ITFConversion
```

```
{
```

```
    public:
```

```
        int prec(char c)
```



```

{
    if(c == '+' || c == '-')
    {
        return 1;
    }
    if(c == '*' || c == '/')
    {
        return 2;
    }
}

```

bool isOperator(char c)

```

{
    if(c == '+' || c == '-' || c == '*' || c == '/')
    {
        return true;
    }
    else
    {
        return false;
    }
}

```

string infixToPostfix(string s)

```

{
    stack<char> st;
    string res;
    for(int i=0;i<s.length();i++)
    {
        if(!isOperator(s[i]))
        {
            res = res+s[i];
            continue;
        }
        if(st.empty())
        {
            st.push(s[i]);
        }
        else
        {
            while(!st.empty() && prec(st.top())>=prec(s[i]))
            {
                res = res + st.top();
                st.pop();
            }
            st.push(s[i]);
        }
    }
    while(!st.empty())
    {
        res = res + st.top();
    }
}

```

```

        st.pop();
    }
    return res;
}

};

int main()
{
    ITFConversion itf;
    string exp ;
    cout<<"Enter expression ...";
    cin>>exp;
    cout<<"Postfix form ..."<<itf.infixToPostfix(exp)<<endl;
    return 0;
}

```

Postfix Notation (Reverse Polish Notation)

Postfix notation is also known as reverse polish notation. It is a mathematical notation in which operators come after their operands. For example, the infix expression $3 + 4$ can be written in postfix notation as $3\ 4\ +$. Similarly, the infix expression $(2 + 3) * 4$ can be written in postfix notation as $2\ 3\ +\ 4\ *$. Postfix notation has several advantages over infix notation. It eliminates the need for parentheses and makes parsing and evaluation of expressions easier.

Postfix Evaluation Algorithm

Postfix evaluation algorithm is a simple algorithm that allows us to evaluate postfix expressions. The algorithm uses a stack to keep track of operands and performs arithmetic operations when an operator is encountered. The algorithm can be summarized in the following steps:

First of all, it will Create an empty stack.

After that, it Scan the expression from left to right.

If an operand is encountered, it push it onto the stack.

If an operator is encountered, pop the top two operands from the stack, perform the operation, and push the result back onto the stack.

After that, it Continue scanning the expression until all tokens have been processed.

When the expression has been fully scanned, the result will be the top element of the stack.

Example:

Let's consider the expression $5\ 6\ 7\ +\ *\ 8\ -$. We will evaluate this expression using the postfix evaluation algorithm.

Start scanning the expression from left to right.

Push operand 5 onto the stack.

Push operand 6 onto the stack.

Push operand 7 onto the stack.

Pop operands 7 and 6 from the stack, perform addition, and push the result (13) back onto the stack.

Pop operands 13 and 5 from the stack, perform multiplication, and push the result (65) back onto the stack.

Push operand 8 onto the stack.

Pop operands 8 and 65 from the stack, perform subtraction, and push the result (57) back onto the stack.

The final result is 57.

C++ Program to Check for Balanced Brackets in an expression (well-formedness) using Stack

```
#include <iostream>
#include<stack>
using namespace std;
class BracesBalance
{
    public:

    bool areBracesBalanced(string str)
    {
        stack<char> st;
        for (int i = 0; i < str.length(); i++)
        {
            char c = str[i];
            if (st.empty())
                st.push(c);

            else if (st.top()=='(' && c == ')' || st.top() == '{' && c == '}' || st.top() == '[' && c == ']')
                //else if (st.top() == ')' && c == '(' || st.top() == '}' && c == '{' || st.top() == ']' && c == '[')

                st.pop();

            else
                st.push(c);
        }
        if (st.empty())
            return true;
        else
            return false;
    }
};
int main()
{
    BracesBalance bb;
    string parenthesis;
    cout<<"Enter set of braces ...";
    cin>>parenthesis;

    if (bb.areBracesBalanced(parenthesis))
        cout << "Braces are balanced";
    else
        cout << "Braces are NOT balanced";
}
```

```
        return 0;
    }
```

C++ program that determines the given string is a Palindrome

```
#include<iostream>
#include<stack>
using namespace std;

bool isPalindrome(string s)
{
    int length = s.size();
    stack<char> st;
    int i, mid = length / 2;
    for (i = 0; i < mid; i++)
    {
        st.push(s[i]);
    }

    if (length % 2 != 0)
    {
        i++;
    }
    char ele;

    while (s[i] != '\0')
    {
        ele = st.top();
        st.pop();

        if (ele != s[i])
            return false;
        else
        {
            i++;
            return true;
        }
    }
}

int main()
{
    string s;
    cout << "Enter the string ...";
    cin>>s;

    if (isPalindrome(s))
        cout << "String is palindrome";
}
```

```

else
    cout << "String is NOT palindrome";

return 0;
}

```

Definition of Queue Data Structure

Queue is a linear data structure that operates elements FIFO (First In First Out) or LILO (Last In Last Out) pattern.

It is named as queue as it behaves like real world queue. Example people waiting in a line to get a ticket.

Queue is an abstract data type with a bounded (predefined) capacity.

It is a simple data structure that allows to enter elements in the rear end also called enqueue operation and elements from the front end also called dequeue operation.



Standard Operations

Enqueue: Elements are added into the queue from rear end.

Dequeue: Elements are removed from the front end.

isFull: Checks if the queue is full or not.

isEmpty: Checks if the queue is empty or not.

Count: Finds the number of elements in the queue.

<pre> isEmpty () { if (front == -1 && rear== -1) return true; else return false; } isFull () { if (rear == size(arr) - 1) return true; else return false; } </pre>	<pre> enqueue(value) { If (isFull ()) return; } else if (isEmpty) { front = rear = 0; } else { rear++; } arr [rear] = value; } </pre>	<pre> Dequeue () { int temp; if (isEmpty ()) return; } else if() { temp = arr(front); front = rear = -1; } else { temp = arr [front]; front++; } </pre>
--	---	---

		return temp; }
--	--	-------------------

Application of Queue data structure

- Is used in disk scheduling, CPU scheduling.
- Handling of interrupts in real-time systems. The interrupts are handled in same order as they arrive first come first served (FIFO).
- In real time call centre phone call systems will use queues to hold people calling them in an order until a service representative is free.
- When data is transferred asynchronously between two processes, queue is used for synchronization.

Queue Implementation using Arrays

A queue is a fundamental data structure that follows the First In, First Out (FIFO) principle. Implementing a queue using arrays involves using a fixed-size array to hold elements and two pointers, front and rear, to keep track of the front and rear of the queue. Elements are added (enqueued) at the rear and removed (dequeued) from the front.

Key Operations - Enqueue

Adds an element to the rear of the queue.

The rear pointer is incremented to the next available position.

Dequeue

Removes the element from the front of the queue.

The front pointer is incremented to the next available position.

isEmpty

Checks if the queue is empty by comparing the front and rear pointers.

isFull

Checks if the queue is full, especially in a fixed-size array.

Peek

Retrieves the element at the front of the queue without removing it.

Algorithm - Initialization:

Create an array with a fixed size to hold the elements.

Initialize front and rear pointers to -1 to indicate an empty queue.

Enqueue Operation

Check if the queue is full.

If not, increment the rear pointer and add the element at the new rear position.

Dequeue Operation:

Check if the queue is empty.

If not, retrieve the element at the front position, increment the front pointer, and return the element.

isEmpty Operation

Return true if front is -1, indicating an empty queue; otherwise, return false.

isFull Operation:

Check if the next position of rear is equal to front, indicating a full queue.

Peek Operation

Return the element at the front position without removing it.

C++ Program that implements to Queue using arrays

```
#include <iostream>
```

```
#define MAX 3
```

```
using namespace std;
```

```
class Queue
```

```
{
```

```
    private:
```

```
        int myqueue[MAX], front, rear;
```

```
    public:
```

```
        Queue()
```

```
        {
```

```
            front = -1;
```

```
            rear = -1;
```

```
        }
```

```
        bool isFull()
```

```
        {
```

```
            if(front == 0 && rear == MAX - 1)
```

```
                return true;
```

```
            else
```

```
                return false;
```

```
        }
```

```
        bool isEmpty()
```

```
        {
```

```
            if(front == -1)
```

```
                return true;
```

```
            else
```

```
                return false;
```

```
        }
```

```
        void enQueue(int value)
```

```
        {
```

```
            if(isFull())
```

```
            {
```

```
                cout << endl<< "Queue is full!!";
```

```
            }
```

```
            else if (isEmpty())
```

```
            {
```

```

    rear = 0;
    front = 0;
    myqueue[rear] = value;
}
else
{
    rear ++;
    myqueue[rear] = value;
}

}

void enqueue(int val)
{
    if (isFull())
    {
        cout << "Queue full" << endl;
        return;
    }
    else if (isEmpty())
    {
        rear = 0;
        front = 0;
        myqueue[rear] = val;
    }
    else
    {
        rear++;
        myqueue[rear] = val;
    }
}

int deQueue()
{
    int value;
    if(isEmpty())
    {
        cout << "Queue is empty!!" << endl;
        return(-1);
    }
    else
    {
        value = myqueue[front];
        if(front >= rear)
        {
            //only one element in queue
            front = -1;
            rear = -1;
        }
        else

```


$$\}$$
$$\}$$
 $\}.$ $\{$

```

        }
    }
    return 0;
}

```

OUTPUT

1.Enqueue 2.Dequeue 3.display 4. Exit

Enter one option 1

Enter one element ... 22

1.Enqueue 2.Dequeue 3.display 4. Exit

Enter one option 1

Enter one element ... 55

1.Enqueue 2.Dequeue 3.display 4. Exit

Enter one option 1

Enter one element ... 88

1.Enqueue 2.Dequeue 3.display 4. Exit

Queue Implementation using Linked Lists

Implementing a queue using linked lists involves using a linked list data structure to manage the elements in the queue. Unlike arrays, linked lists allow for dynamic memory allocation, making it easier to manage the queue's size dynamically. In this implementation, a linked list node represents each element in the queue.

Key Operations

Enqueue

Adds an element to the rear of the queue.

The new element is added as the last node in the linked list.

Dequeue Removes the element from the front of the queue.

The front node of the linked list is removed.

isEmpty

Checks if the queue is empty by examining if the linked list is empty.

Peek

Retrieves the element at the front of the queue without removing it.

Algorithm - Initialization:

Create an empty linked list to represent the queue.

Maintain references to the front and rear nodes.

Enqueue Operation

Create a new node with the given data.

If the queue is empty, set both the front and rear references to the new node.
Otherwise, update the rear node's next reference to the new node and update the rear reference to the new node.

Dequeue Operation

If the queue is empty, raise an exception or return an indication of an empty queue.

Retrieve the data from the front node.

If the front node is also the rear node (single node in the queue), set both front and rear references to None.

Otherwise, update the front reference to the next node.

isEmpty Operation

Return true if both front and rear references are None, indicating an empty queue; otherwise, return false.

Peek Operation:

If the queue is empty, raise an exception or return an indication of an empty queue.

Return the data from the front node without removing it.

Implementation of queue using linked lists

```
#include <iostream>
using namespace std;
```

```
class Node
{
public:
    int data;
    Node *next;

    Node(int value)
    {
        data = value;
        next = NULL;
    }
};

class Queue
{
private:
    Node* front;
    Node* rear;

public:
    Queue()
    {
        front = NULL;
        rear = NULL;
    }
};
```

```

void enqueue(int item)
{
    Node* newNode = new Node(item);

    if (front == NULL)
    {
        front = rear = newNode;
    }
    else
    {
        rear->next = newNode;
        rear = newNode;
    }
}

void dequeue()
{
    if (front == NULL)
    {
        cout << "Queue is empty"<<endl;
        return;
    }

    Node* temp = front;

    cout << "Dequeued from the queue ..." << temp->data << endl;
    front = front->next;
    delete temp;

    if (front == NULL)
    {
        rear = NULL;
    }
}

void display()
{
    if (front == NULL)
    {
        cout << "Queue is empty"<<endl;
        return;
    }

    cout << "Queue elements ... ";
    Node* temp = front;
    while (temp != NULL)
    {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
}

```

```

        }
        cout << endl;
    }
};

int main()
{
    Queue queue;

    int option, element;

    cout << "1. Enqueue"<<endl;
    cout << "2. Dequeue"<<endl;
    cout << "3. Display"<<endl;
    cout << "4. Exit"<<endl;
    do
    {
        cout << "Enter an option ...";
        cin >> option;

        switch (option)
        {
            case 1:
                cout << "Enter an element ...";
                cin >> element;
                queue.enqueue(element);
                break;

            case 2:
                queue.dequeue();
                break;

            case 3:
                queue.display();
                break;

            case 4:
                exit(0);

            default:
                cout << "Enter correct option"<<endl;
        }
    } while (option != 4);

    return 0;
}

```

Circular Queue

<pre>isEmpty () { if (front == -1 && rear == -1) return true; else return false; } isFull () { if ((rear + 1) % n == front) return true; else return false; }</pre>	<pre>enqueue(value) { If (is Full ()) return; else if (isEmpty) { front = rear = 0; } else rear = (rear + 1) % n; arr [rear] = value; }</pre>	<pre>Dequeue () { int temp; if (isEmpty ()) return; else if () { temp = arr(front); front = rear = -1; } else { temp = arr [front]; front = (front + 1) % n; } return temp; }</pre>
---	---	---

A C++ program on implementing Circular Queue

```
#include<iostream>
```

```
using namespace std;
```

```
class CircularQueue
```

```
{
    private:
        int front;
        int rear;
        int arr[5];
        int itemCount;

    public:
        CircularQueue()
        {
            itemCount = 0;
            front = -1;
            rear = -1;
            for (int i = 0; i < 5; i++)
            {
                arr[i] = 0;
            }
        }
}
```

```
bool isEmpty()
{
    if (front == -1 && rear == -1)
```

```

        return true;
    else
        return false;
}
bool isFull()
{
    if ((rear + 1) % 5 == front)
        return true;
    else
        return false;
}

```

```

void enqueue(int val)
{
    if (isFull())
    {
        cout << "\nQueue full" << endl;
        return;
    } else if (isEmpty())
    {
        rear = 0;
        front = 0;
        arr[rear] = val;

    } else
    {
        rear = (rear + 1) % 5;
        arr[rear] = val;

    }
    itemCount++;
}

```

```

int dequeue()
{
    int x = 0;
    if (isEmpty())
    {
        cout << "\nQueue is Empty" << endl;
        return x;
    }
    else if (rear == front)
    {
        x = arr[rear];
        rear = -1;
        front = -1;
        itemCount--;
        return x;
    }
}

```

```

    }
else
{
    cout << "\nFront value: " << front << endl;
    x = arr[front];
    arr[front] = 0;
    front = (front + 1) % 5;
    itemCount--;
    return x;
}
}

int count()
{
    return (itemCount);
}

void display()
{
    cout << "\nElements in the Queue are ..." << endl;
    for (int i = 0; i < 5; i++)
    {
        cout << arr[i] << " ";
    }
}

};

int main()
{
    CircularQueue cq;
    int value, option;

    do
    {
        cout << "\n\nSelect Option number to continue or Enter 0 to exit." << endl;

        cout << "1. Enqueue()" << endl;
        cout << "2. Dequeue()" << endl;
        cout << "3. isEmpty()" << endl;
        cout << "4. isFull()" << endl;
        cout << "5. count()" << endl;
        cout << "6. display()" << endl;
        cout << "7. Clear Screen" << endl << endl;

        cin >> option;

        switch (option)
        {
            case 0:

```



```

        break;
    case 1:
        cout << "\nEnter an Element to Enqueue in the Queue" << endl;
        cin >> value;
        cq.enqueue(value);
        break;
    case 2:
        cout << "\nDequeued Value : " << cq.dequeue() << endl;
        break;
    case 3:
        if (cq.isEmpty())
            cout << "\nQueue is Empty" << endl;
        else
            cout << "\nQueue is not Empty" << endl;
        break;
    case 4:
        if (cq.isFull())
            cout << "\nQueue is Full" << endl;
        else
            cout << "Queue is not Full" << endl;
        break;
    case 5:
        cout << "\nNumber of elements in Queue : " << cq.count() << endl;
        break;
    case 6:
        cq.display();
        break;
    case 7:
        system("cls");
        break;
    default:
        cout << "Enter Correct Option number " << endl;
    }
}

} while (option != 0);

return 0;
}

```

OUTPUT

Select Option number to continue or Enter 0 to exit.

1. Enqueue ()
2. Dequeue ()
3. isEmpty ()
4. isFull ()
5. count ()
6. display ()
7. Clear Screen

Enter an Element to Enqueue in the Queue 11

Select Option number to continue or Enter 0 to exit.

- 1. Enqueue ()*
- 2. Dequeue ()*
- 3. isEmpty ()*
- 4. isFull ()*
- 5. count ()*
- 6. display ()*
- 7. Clear Screen*

1

Enter an Element to Enqueue in the Queue 22

Select Option number to continue or Enter 0 to exit.

- 1. Enqueue ()*
- 2. Dequeue ()*
- 3. isEmpty ()*
- 4. isFull ()*
- 5. count ()*
- 6. display ()*
- 7. Clear Screen*

Real-time applications of queue

Telecommunications

Call Centres: customers are served based on a first-come, first-served basis. Calls are placed in a queue, and agents attend to them in the order they are received.

Data Packet Queues: In computer networks, data packets are often queued for transmission. Queuing theory helps optimize the performance of network protocols.

Transportation:

Traffic Flow: Queuing theory is used to model and analyse traffic flow at intersections and on highways. It helps in optimizing traffic signal timings and managing congestion.

Airport Security: Passengers waiting in security lines at airports are essentially forming a queue. Understanding queue dynamics helps optimize security processes and reduce wait times.

Manufacturing and Operations:

Production Lines: In manufacturing, queue theory is used to optimize production lines by minimizing wait times and improving efficiency in the movement of materials.

Service Operations: In service-oriented operations, such as banks or hospitals, queuing theory is applied to optimize the flow of customers and minimize waiting times.

Computer Science:

Task Scheduling: In operating systems, queuing theory is applied to task scheduling. Processes in a computer system wait in a queue for the CPU's attention.

Print Job Queues: Printers often have queues to manage multiple print jobs. Queuing theory helps optimize the order in which print jobs are processed.

Finance:

Stock Exchange: Orders placed in a stock exchange form a queue. Understanding queuing theory helps optimize order processing and reduce transaction times.

Bank Transactions: In banking, customers waiting in line for transactions form a queue. Queuing theory helps optimize the number of tellers and service times.

Vaccine Distribution: During mass vaccination campaigns, queuing theory can be used to optimize the distribution of vaccines and manage the flow of people.

Retail:

Checkout Lines: Queuing theory is applied to optimize the number of checkout counters open at a retail store, minimizing wait times for customers.

Supply Chain: Queuing theory is used in supply chain management to optimize the flow of goods through various stages of production and distribution.

Entertainment:

Amusement Parks: Queuing theory is applied to optimize ride queues, ensuring that visitors have an enjoyable experience with minimal waiting times.

Online Streaming: In video streaming services, queuing theory is used to manage the distribution of content efficiently, reducing buffering times.

Networking and Message Queues

Networking, message queues play a crucial role in facilitating communication and coordination between distributed systems. A message queue is a form of inter-process communication that enables asynchronous communication between various components of a networked system. Here's the theory behind the use of message queues in networking:

Asynchronous Communication

Definition: Asynchronous communication refers to a communication model where messages can be sent and received independently of each other. In networking, this is crucial for systems with varying processing speeds or delays.

Application in Message Queues: Message queues allow components to send and receive messages asynchronously. Producers can enqueue messages without waiting for consumers to process them immediately.

Load Balancing

Definition: Load balancing involves distributing workloads across multiple servers to ensure optimal resource utilization and prevent overload on any single server.

Application in Message Queues: Message queues can be used to implement load balancing. Work units are enqueued and distributed among available workers, ensuring a more balanced distribution of tasks.

Fault Tolerance

Definition: Fault tolerance is the ability of a system to continue operating in the presence of faults or failures.

Application in Message Queues: Message queues contribute to fault tolerance by providing a buffer for messages. If a component fails temporarily, the messages remain in the queue, and the system can recover gracefully when the component is restored.

Scalability

Definition: Scalability is the ability of a system to handle an increasing amount of workload or to be easily expanded.

Application in Message Queues: Message queues support scalability by allowing for the distribution of tasks among multiple components. As the load increases, additional consumers can be added to process messages concurrently.

Protocol Flexibility

Definition: Protocol flexibility refers to the ability of systems using different communication protocols to interact seamlessly.

Application in Message Queues: Message queues provide a middleware layer that abstracts the communication between components. This abstraction allows systems using different protocols or technologies to exchange messages without direct coupling.

Message Persistence

Definition: Message persistence refers to the ability of a system to store messages even in the face of system failures or restarts.

Application in Message Queues: Message queues often support message persistence, ensuring that important messages are not lost in the event of system failures. Persisted messages can be retrieved when the system is back online.

Quality of Service (QoS):

Definition: Quality of Service refers to the level of service provided by a network in terms of factors like reliability, latency, and throughput.

Application in Message Queues: Message queues can be configured to provide specific Quality of Service levels. This includes defining priorities for messages, managing delivery guarantees, and controlling message expiration.

Security

Definition: Security is a critical aspect of networked systems, ensuring that communication remains confidential, authentic, and protected against unauthorized access.

Application in Message Queues: Message queues often incorporate security features such as encryption, authentication, and access controls to safeguard the communication between components.

Real-Time Processing

Definition: Real-time processing involves handling and responding to events or data as they occur in near real-time.

Application in Message Queues: Message queues can facilitate real-time processing by enabling components to react to events as soon as they are enqueued.

Data Flow Control

Definition: Data flow control involves managing the rate at which data is transferred between components to avoid congestion or overload.

Application in Message Queues: Message queues provide a mechanism for data flow control. Producers can enqueue messages at their own pace, and consumers can process them based on their capacity, preventing bottlenecks.

==00oo00==